

SESIÓN 01 | INTRODUCCIÓN AL DISEÑO DE SISTEMAS

UNIDAD DE APRENDIZAJE 1 - UA1

- Conceptos de diseño de sistemas
- Buenas prácticas de diseño de sistemas
- Diseño de la arquitectura del sistema

El diseño de sistemas de información es una parte fundamental del desarrollo de sistemas de información. Es el proceso que toma los requisitos del sistema, definidos en el análisis de sistemas, y los convierte en una especificación detallada de cómo se construirá el sistema.

Entonces, durante esta sesión:

Conocerás los principales conceptos de diseño de sistemas.

Aprenderás a usar buenas prácticas de diseño de sistemas en un proyecto de desarrollo de sistemas.

Comprenderás el alcance del diseño de la arquitectura del sistema.

/ CONCEPTOS DE DISEÑO DE SISTEMAS

/ CONCEPTOS DE DISEÑO DE SISTEMAS

EL DISEÑO DE SISTEMAS

- ❑ El diseño de sistemas es como el “plano” en la construcción de un edificio. Si tienes un diseño sólido desde el principio, te ahorrarás muchos problemas más adelante.
- ❑ En esta fase, se toman las especificaciones y requisitos identificados y se traducen en una “arquitectura” que los desarrolladores pueden implementar.
- ❑ Aquí es donde se toman decisiones sobre cómo se organizarán los datos, cómo se procesarán y cómo interactúan los diferentes componentes del sistema.



ARQUITECTURA DEL SISTEMA

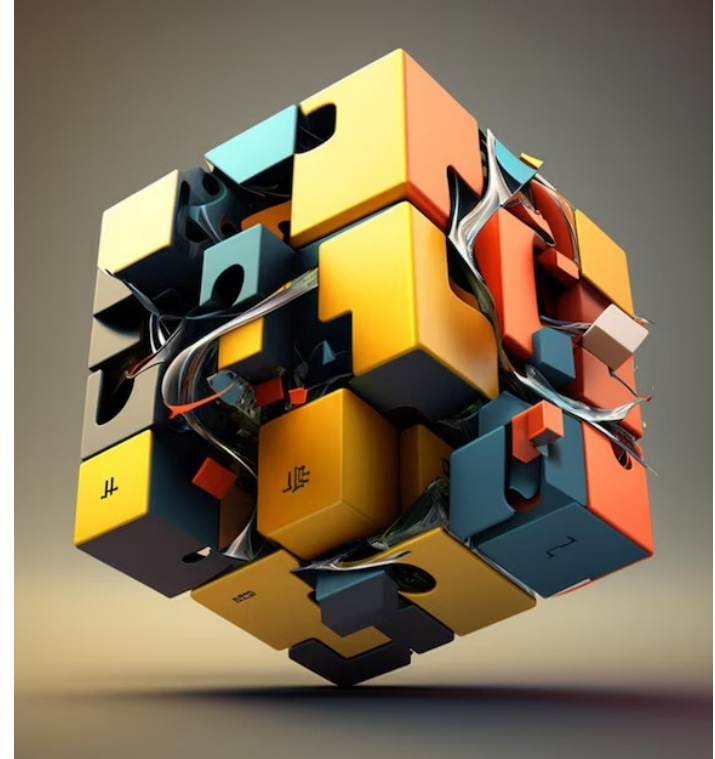
- ❑ Es el esquema de alto nivel que muestra cómo se organizan los diferentes **componentes del software** y cómo interactúan entre sí.
- ❑ Incluye la organización de módulos, la asignación de responsabilidades y la definición de interfaces y comunicaciones entre componentes.
- ❑ Ejemplo: Imagina que estás construyendo un sistema de comercio electrónico. La arquitectura podría incluir un frontend para la interacción del usuario, un backend para el procesamiento de la lógica empresarial, una base de datos para almacenar información del producto y del usuario, y una API para manejar pagos.



/ CONCEPTOS DE DISEÑO DE SISTEMAS

DESCOMPOSICIÓN MODULAR

- ❑ La descomposición modular implica **dividir un sistema complejo en partes más pequeñas y manejables**, llamadas módulos.
- ❑ Cada módulo se encarga de una funcionalidad específica y opera de manera independiente tanto como sea posible.
- ❑ Ejemplo: *En el mismo sistema de comercio electrónico, podrías tener módulos separados para manejar la autenticación de usuarios, el carrito de compras, la gestión de productos, y el procesamiento de pagos. Cada uno de estos módulos se desarrolla y se mantiene de forma independiente pero trabajan juntos para formar el sistema completo.*



ABSTRACCIÓN

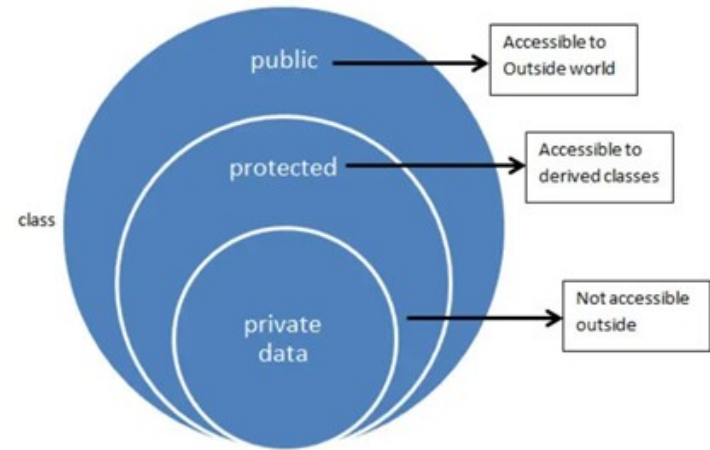
- ❑ La abstracción es el concepto de ignorar los detalles menos importantes para **centrarse en las características esenciales**.
- ❑ Esto permite una comprensión más clara del sistema y facilita su diseño y mantenimiento.
- ❑ Ejemplo: Cuando usas un auto, no necesitas saber cómo funciona exactamente el motor para poder conducir. El funcionamiento interno del motor es una abstracción que te permite centrarte en la tarea de conducir.



/ CONCEPTOS DE DISEÑO DE SISTEMAS

ENCAPSULAMIENTO

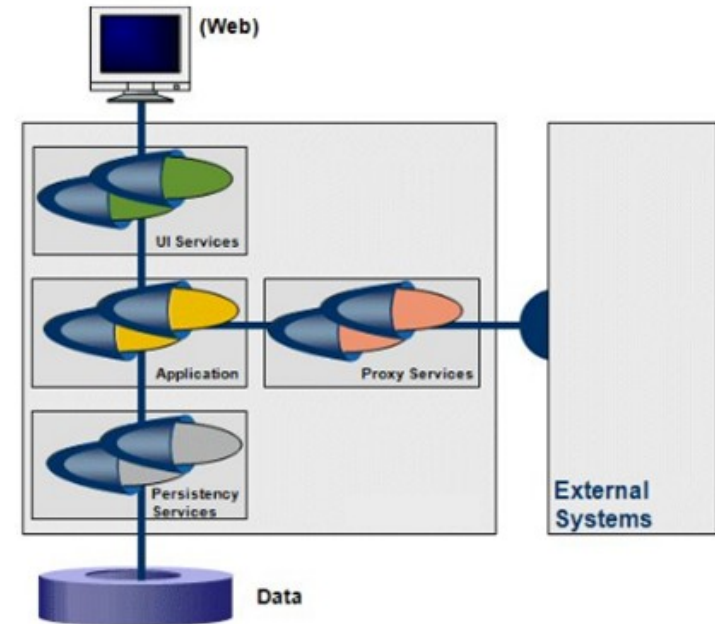
- ❑ El encapsulamiento es una técnica de diseño de software que implica **ocultar los detalles internos de un módulo** y exponer sólo las operaciones que son seguras y relevantes para el resto del sistema.
- ❑ Esto mejora la seguridad y la integridad de los datos.
- ❑ Ejemplo: En una aplicación bancaria, el módulo que maneja las transacciones financieras encapsulan los detalles de cómo se realizan las transferencias de dinero entre cuentas. Solo expondrá métodos como `depositar()`, `retirar()` y `transferir()`, asegurando que los detalles internos no sean accesibles para el resto del sistema.



/ CONCEPTOS DE DISEÑO DE SISTEMAS

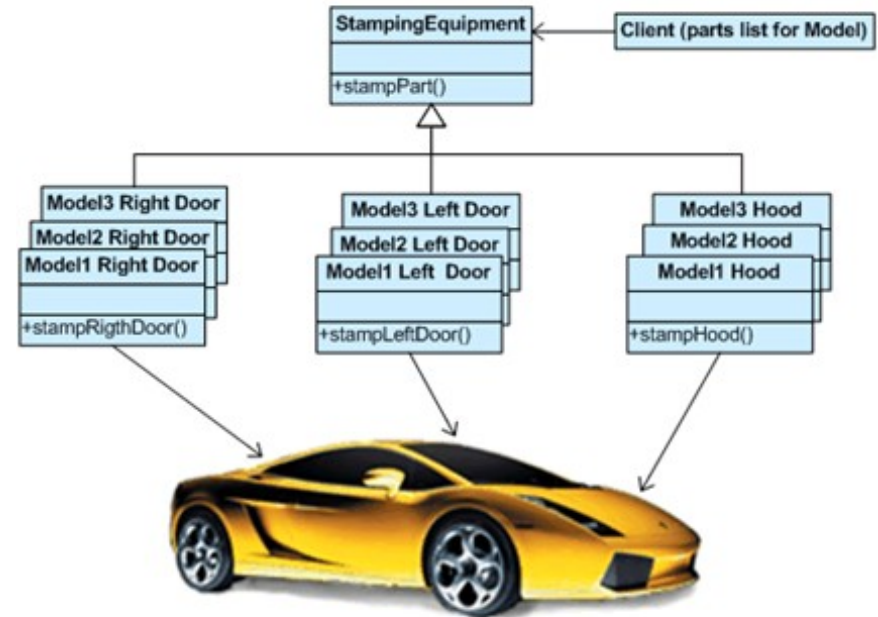
SEPARACIÓN DE CONCERNS

- ❑ La “Separación de Concerns” se refiere a la práctica de **dividir un programa en distintas secciones**, cada una de las cuales aborda un “concern” (preocupación o aspecto) específico.
- ❑ Esto facilita la modularidad y hace que el código sea más fácil de entender, mantener y escalar.
- ❑ Ejemplo: En una aplicación web, podrías separar los “concerns” en tres capas principales: la capa de presentación (frontend), la capa de lógica de negocio (backend) y la capa de datos (base de datos). Cada capa tiene responsabilidades específicas y no se mezclan entre sí.



PATRONES DE DISEÑO

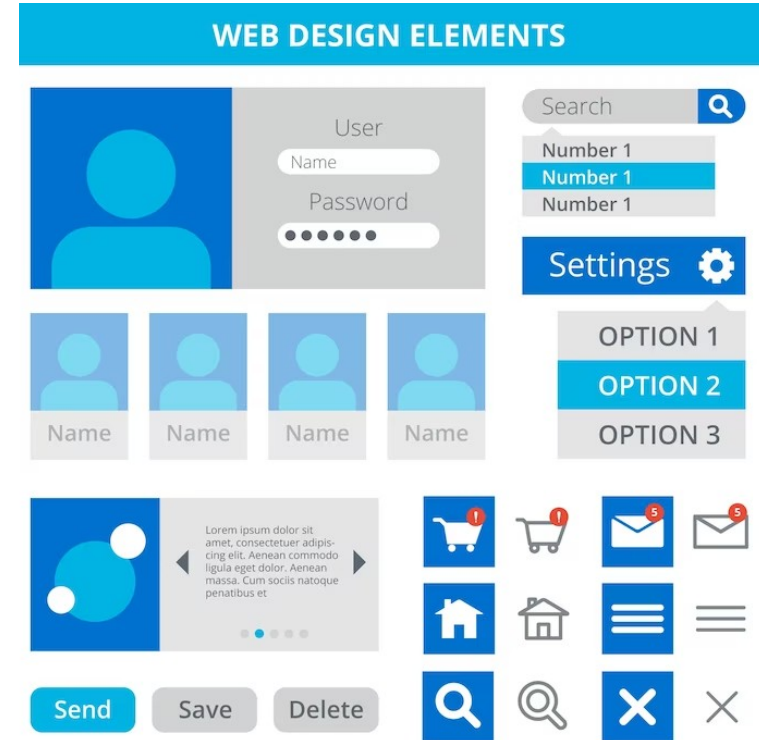
- ❑ Los patrones de diseño **son soluciones probadas a problemas comunes** que se encuentran en el diseño de software.
- ❑ Estos patrones ofrecen una estructura que puede ayudar a resolver problemas similares de manera más eficiente.
- ❑ Ejemplo: El patrón “Singleton” asegura que una clase tenga solo una instancia y proporciona un punto de acceso global a ella. Esto es útil, por ejemplo, para un gestor de configuración en una aplicación, donde solo quieres una única instancia que maneje todas las configuraciones.



/ CONCEPTOS DE DISEÑO DE SISTEMAS

INTERFAZ / EXPERIENCIA DE USUARIO

- ❑ **UI (Interfaz de Usuario):** Se refiere al diseño de las interfaces a través de las cuales los usuarios interactúan con una aplicación.
- ❑ **UX (Experiencia de Usuario):** Es más amplio y se refiere a la experiencia general que un usuario tiene al interactuar con un sistema, incluido el diseño, la utilidad y la eficiencia.
- ❑ **Ejemplo:** Supongamos que estás diseñando una aplicación móvil para reservar vuelos. La UI incluiría el diseño de los botones, los campos de texto y los menús. La UX, por otro lado, consideraría cómo el usuario encuentra el vuelo que desea, cómo se siente durante el proceso y cuán fácil es completar la reserva.



/ CONCEPTOS DE DISEÑO DE SISTEMAS

BASE DE DATOS

- ❑ La base de datos es **donde se almacena la información** que maneja una aplicación.
- ❑ El diseño de la base de datos implica decidir qué datos se almacenarán, cómo se organizarán y cómo se accederá a ellos.
- ❑ Ejemplo: En un sistema de gestión de bibliotecas, la base de datos podría tener tablas para libros, usuarios, préstamos y multas. Cada tabla tendría campos específicos; por ejemplo, la tabla de libros podría tener campos como ISBN, título, autor y estado (disponible o prestado).



SEGURIDAD

- ❑ La seguridad en el diseño de sistemas implica tomar medidas para **proteger** la integridad, disponibilidad y confidencialidad de los datos y servicios del sistema.
- ❑ Esto incluye aspectos como autenticación, autorización, cifrado y auditoría.
- ❑ Ejemplo: En una aplicación de banca en línea, la seguridad es crítica. Podrías implementar autenticación de dos factores para garantizar que solo los usuarios autorizados tengan acceso a las cuentas. Además, todas las transacciones podrían cifrarse para proteger la confidencialidad de los datos.



/ CONCEPTOS DE DISEÑO DE SISTEMAS

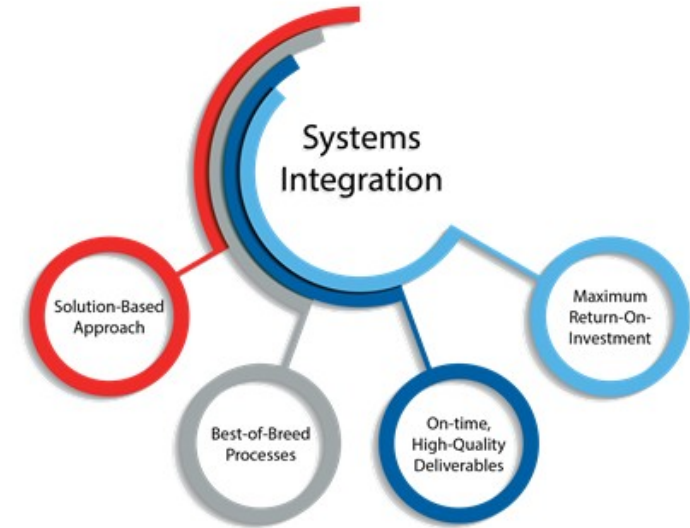
ESCALABILIDAD Y RENDIMIENTO

- ❑ **Escalabilidad:** Se refiere a la capacidad del sistema para manejar un aumento en la carga de trabajo o demanda de manera efectiva.
- ❑ **Rendimiento:** Es la eficiencia con la que un sistema realiza sus tareas bajo una carga determinada.
- ❑ **Ejemplo:** Si tienes un sitio web de comercio electrónico que recibe un alto tráfico durante las temporadas de vacaciones, deberías diseñar el sistema para que pueda escalar rápidamente, añadiendo más servidores según sea necesario. El rendimiento podría optimizarse mediante el uso de cachés y balanceadores de carga.



INTEGRACIÓN DE SISTEMAS

- ❑ La integración de sistemas implica conectar diferentes sistemas informáticos para que funcionen como una unidad cohesiva.
- ❑ Esto puede incluir la integración de bases de datos, aplicaciones y otros recursos.
- ❑ Ejemplo: En un hospital, podrías tener un sistema de registro de pacientes, otro para la gestión de citas y un tercero para la facturación. La integración efectiva de estos sistemas permitiría que un cambio en el registro de un paciente se refleje automáticamente en los otros sistemas relacionados.



DOCUMENTACIÓN

- ❑ La documentación es el conjunto de materiales escritos que explican cómo funciona un sistema, cómo mantenerlo y cómo usarlo.
- ❑ Esto es crucial tanto para los desarrolladores como para los usuarios finales.
- ❑ Ejemplo: En un proyecto de software empresarial, la documentación podría incluir manuales de usuario, documentación técnica para desarrolladores, diagramas de arquitectura y guías de instalación y configuración.



/ BUENAS PRÁCTICAS DE DISEÑO DE SISTEMAS

/ BUENAS PRÁCTICAS DE DISEÑO DE SISTEMAS

¿EN QUÉ CONSISTEN?

- ❑ Son un conjunto de métodos y técnicas que se han demostrado eficaces para crear sistemas de alta calidad. Estas prácticas se basan en la experiencia de los profesionales del diseño de sistemas y en la investigación académica.
- ❑ Abarcan todos los aspectos del proceso de diseño, desde la definición de los requisitos hasta la implementación y el mantenimiento del sistema.



/ BUENAS PRÁCTICAS DE DISEÑO DE SISTEMAS

DISEÑO MODULAR

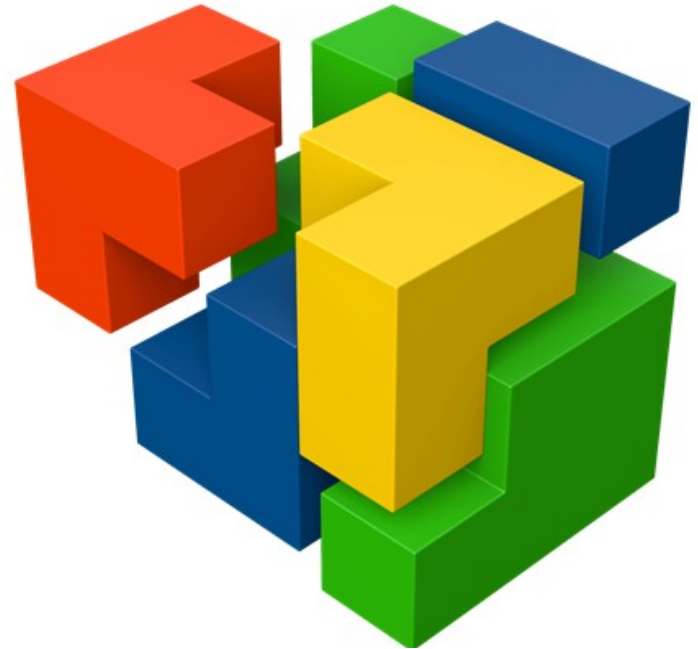
¿En qué consiste?

- ❑ Dividir el sistema en módulos más pequeños y manejables que puedan desarrollarse y mantenerse de manera independiente.

¿Por qué es una buena práctica?

- ❑ Facilita la mantenibilidad, permite la reutilización de código y hace que el sistema sea más fácil de entender.

Ejemplo: En un sistema de gestión de inventario, podrías tener módulos separados para la gestión de productos, la gestión de proveedores y la gestión de pedidos. Cada módulo se encargaría de una parte específica del sistema general.



/ BUENAS PRÁCTICAS DE DISEÑO DE SISTEMAS

PRINCIPIO DE RESPONSABILIDAD ÚNICA

¿En qué consiste?

- ❑ Cada módulo o componente del sistema debe tener una única responsabilidad o función.

¿Por qué es una buena práctica?

- ❑ Reduce la complejidad, mejora la legibilidad del código y facilita las pruebas y el mantenimiento.

Ejemplo: En una aplicación de correo electrónico, podrías tener un componente dedicado exclusivamente a la gestión de la bandeja de entrada, otro para el envío de correos y otro para la gestión de contactos.



/ BUENAS PRÁCTICAS DE DISEÑO DE SISTEMAS

ABSTRACCIÓN Y ENCAPSULAMIENTO

¿En qué consiste?

- ❑ Ocultar los detalles de implementación y exponer sólo las interfaces necesarias para interactuar con un módulo o componente.

¿Por qué es una buena práctica?

- ❑ Mejora la seguridad y permite cambios en el código subyacente sin afectar a otros componentes del sistema.

Ejemplo: En un sistema de pago en línea, el módulo de pagos podría ofrecer una interfaz simple como **realizar Pago (monto, moneda)** que oculta la complejidad de interactuar con diferentes proveedores de servicios de pago.



/ BUENAS PRÁCTICAS DE DISEÑO DE SISTEMAS

USO DE PATRONES DE DISEÑO

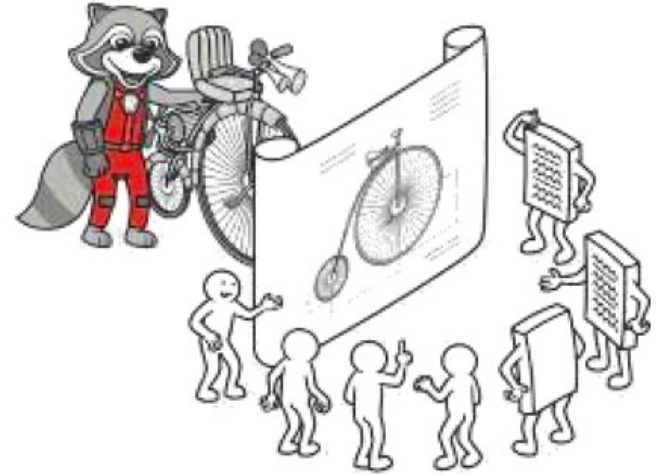
¿En qué consiste?

- ❑ Aplicar patrones de diseño probados para resolver problemas comunes en el diseño de software.

¿Por qué es una buena práctica?

- ❑ Los patrones de diseño son soluciones probadas que pueden aumentar la eficiencia del desarrollo y facilitar la resolución de problemas complejos.

Ejemplo: Si estás construyendo un sistema de notificaciones, podrías usar el patrón **Observer** para permitir que diferentes partes del sistema se suscriban a eventos de notificación sin tener que conocer los detalles de cómo se generan esas notificaciones.



/ BUENAS PRÁCTICAS DE DISEÑO DE SISTEMAS

DISEÑO ORIENTADO A LA ESCALABILIDAD

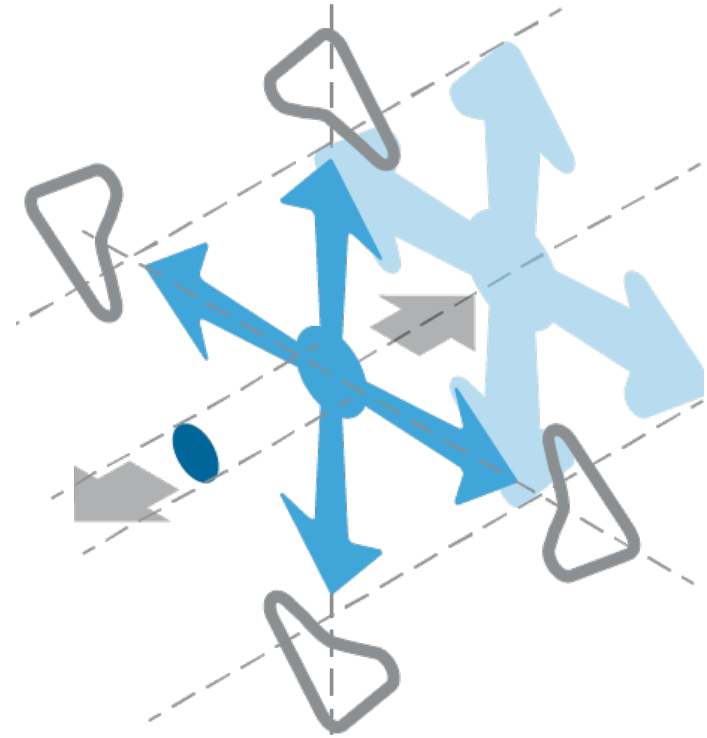
¿En qué consiste?

- ❑ Diseñar el sistema para que pueda manejar un aumento en la carga de trabajo sin afectar negativamente el rendimiento.

¿Por qué es una buena práctica?

- ❑ Prepara al sistema para el crecimiento futuro, evitando costosos rediseños o refactorizaciones.

Ejemplo: En una red social, podrías diseñar la arquitectura para que los servidores se puedan añadir o quitar dinámicamente en función de la carga de trabajo, utilizando técnicas como balanceo de carga y microservicios.



/ BUENAS PRÁCTICAS DE DISEÑO DE SISTEMAS

ENFOQUE EN LA SEGURIDAD DESDE EL DISEÑO

¿En qué consiste?

- ❑ Incorporar consideraciones de seguridad desde las primeras etapas del diseño, en lugar de añadirlas como una idea posterior.

¿Por qué es una buena práctica?

- ❑ Reduce la vulnerabilidad del sistema a ataques y mejora la confianza del usuario.

Ejemplo: En una aplicación de almacenamiento en la nube, podrías implementar el cifrado de extremo a extremo desde el inicio, asegurando que solo los usuarios puedan acceder a sus propios datos.



/ BUENAS PRÁCTICAS DE DISEÑO DE SISTEMAS

DOCUMENTACIÓN EXHAUSTIVA

¿En qué consiste?

- ❑ Crear documentación detallada que explique el diseño, las decisiones tomadas, y cómo interactúan los diferentes componentes.

¿Por qué es una buena práctica?

- ❑ Facilita el mantenimiento y la extensión futura del sistema, y ayuda a nuevos desarrolladores a entender rápidamente el proyecto.

Ejemplo: En un proyecto de código abierto, podrías incluir una documentación completa que abarque desde la instalación y configuración hasta ejemplos de uso y contribución para futuros desarrolladores.



/ BUENAS PRÁCTICAS DE DISEÑO DE SISTEMAS

DISEÑO CENTRADO EN EL USUARIO

¿En qué consiste?

- ❑ Involucrar a los usuarios en el proceso de diseño para asegurarse de que el sistema cumpla con sus necesidades y expectativas.

¿Por qué es una buena práctica?

- ❑ Mejora la adopción del sistema y la satisfacción del usuario, lo que a menudo es crítico para el éxito del proyecto.

Ejemplo: Si estás desarrollando una aplicación móvil para el seguimiento de la salud, podrías realizar entrevistas con usuarios potenciales para entender sus necesidades y luego diseñar la interfaz y las funcionalidades en torno a esos requisitos.



/ BUENAS PRÁCTICAS DE DISEÑO DE SISTEMAS

CONSIDERACIONES

- ❑ Estas buenas prácticas no son sólo teóricas; tienen un impacto real en la calidad del sistema final.
- ❑ Un diseño modular bien ejecutado puede hacer que un sistema sea mucho más fácil de mantener y extender en el futuro. La seguridad desde el diseño no sólo protege la integridad del sistema, sino que también puede evitar costosos incidentes de seguridad.
- ❑ Invertir tiempo en seguir estas buenas prácticas durante la fase de diseño puede ahorrar mucho tiempo y recursos en etapas posteriores del ciclo de vida del desarrollo.



/ DISEÑO DE LA ARQUITECTURA DEL SISTEMA

/ DISEÑO DE LA ARQUITECTURA DEL SISTEMA

¿EN QUÉ CONSISTE?

- ❑ Es una de las etapas más críticas en el desarrollo de software y sistemas de información.
- ❑ En esta fase se define la estructura general del sistema, incluyendo cómo se organizan y se interconectan sus diferentes componentes.
- ❑ Aquí se toman decisiones clave que afectarán a todo el ciclo de vida del proyecto, desde el desarrollo hasta el mantenimiento y la escalabilidad.



/ DISEÑO DE LA ARQUITECTURA DEL SISTEMA

COMPONENTES [1/3]

❑ Componentes y Módulos:

Identificación de las principales piezas funcionales del sistema, que podrían ser módulos de software, microservicios, bibliotecas, etc.

❑ Interconexión:

Definición de cómo estos componentes se comunicarán entre sí. Esto puede implicar el uso de APIs, colas de mensajes, bases de datos compartidas, etc.

❑ Flujo de Datos:

Descripción de cómo los datos se moverán a través del sistema, desde la entrada hasta la salida, pasando por cualquier procesamiento intermedio.



/ DISEÑO DE LA ARQUITECTURA DEL SISTEMA

COMPONENTES [2/3]

❑ Capas de Software:

En sistemas más complejos, los componentes se organizan a menudo en capas que abordan diferentes “concerns”, como la presentación, la lógica de negocio y el acceso a datos.

❑ Escalabilidad y Rendimiento:

Diseño de mecanismos que permitan al sistema manejar un aumento de la carga, como balanceadores de carga, cachés y bases de datos distribuidas.

❑ Seguridad:

Incorporación de características de seguridad, como autenticación, autorización, cifrado y auditoría.



/ DISEÑO DE LA ARQUITECTURA DEL SISTEMA

COMPONENTES [3/3]

❑ Resiliencia y Recuperación:

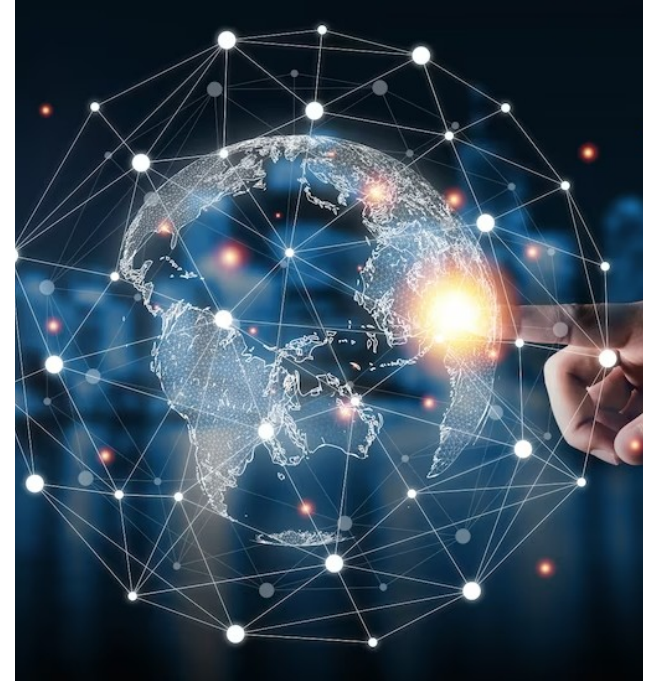
Planificación de cómo el sistema se recuperará de fallos y errores, que puede incluir redundancia, copias de seguridad y protocolos de recuperación.

❑ Interoperabilidad:

Si el sistema debe interactuar con otros sistemas externos, la arquitectura debe diseñarse para facilitar esta integración.

❑ Documentación:

Creación de documentos y diagramas que describen la arquitectura de manera que los desarrolladores, administradores y otras partes interesadas puedan entenderla.



/ DISEÑO DE LA ARQUITECTURA DEL SISTEMA

Imaginemos que estamos diseñando la arquitectura para un sistema de comercio electrónico.

- ❑ **Componentes y Módulos:** Podríamos tener módulos para la gestión de inventario, el carrito de compras, el procesamiento de pagos y la gestión de usuarios.
- ❑ **Interconexión:** Estos módulos podrían comunicarse a través de APIs RESTful.
- ❑ **Flujo de Datos:** Cuando un usuario realiza una compra, los datos podrían fluir desde el módulo del carrito de compras al módulo de procesamiento de pagos y finalmente al módulo de gestión de inventario para actualizar las existencias.
- ❑ **Capas de Software:** Podríamos tener una capa de presentación (frontend), una capa de lógica de negocio (backend) y una capa de datos (base de datos).
- ❑ **Escalabilidad y Rendimiento:** Podríamos usar un balanceador de carga para distribuir las solicitudes de los usuarios entre múltiples servidores.

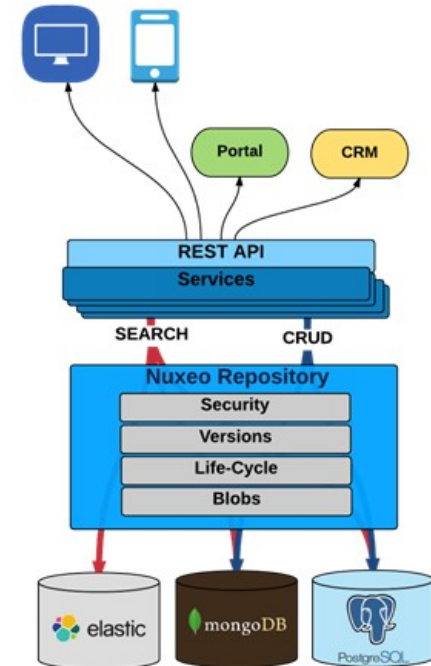
/ DISEÑO DE LA ARQUITECTURA DEL SISTEMA

- ❑ **Seguridad:** Implementaremos autenticación de dos factores para los usuarios y cifrado para las transacciones financieras.
- ❑ **Resiliencia y Recuperación:** Tendríamos copias de seguridad regulares y un plan de recuperación ante desastres.
- ❑ **Interoperabilidad:** Si necesitamos conectar con un sistema de gestión de almacenes, nuestra arquitectura debería facilitar esa integración.

/ DISEÑO DE LA ARQUITECTURA DEL SISTEMA

METODOLOGÍAS

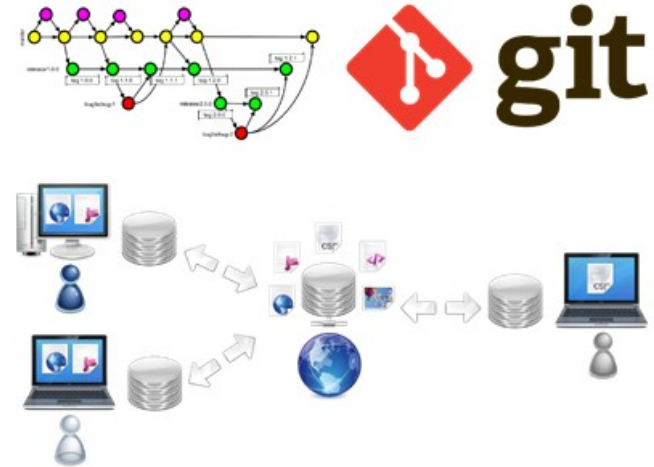
- ❑ **Arquitectura Dirigida por Modelos (MDA):**
Utiliza modelos para definir la arquitectura, lo que facilita la comunicación entre los stakeholders y permite una implementación más eficiente.
- ❑ **Arquitectura Orientada a Servicios (SOA):**
Enfoca el diseño del sistema en torno a servicios independientes que interactúan entre sí.
- ❑ **Microservicios:**
Similar a SOA pero más granular, donde cada servicio es una aplicación independiente que puede ser desarrollada, desplegada y escalada de forma independiente.
- ❑ **Arquitectura en Capas:**
Divide el sistema en capas lógicas (por ejemplo, presentación, lógica de negocio, acceso a datos) para separar responsabilidades.



/ DISEÑO DE LA ARQUITECTURA DEL SISTEMA

HERRAMIENTAS

- ❑ **Enterprise Architect:**
Para modelado y diseño que soporta múltiples frameworks y notaciones.
- ❑ **Microsoft Visio:**
Para crear diagramas de arquitectura.
- ❑ **Archimate:**
Lenguaje de modelado para describir arquitecturas empresariales.
- ❑ **Docker y Kubernetes:**
Para arquitecturas basadas en contenedores y microservicios para el despliegue y la gestión.
- ❑ **Git:**
No es una herramienta de diseño per se, pero es crucial para la gestión de versiones del código y la documentación de la arquitectura.



/ CONCLUSIONES

- Tomar decisiones acertadas en el Diseño de la Arquitectura establece la base para la eficiencia, escalabilidad y seguridad del sistema completo.
- Ignorar principios de diseño sólidos como la modularidad, la responsabilidad única y la seguridad desde el inicio puede llevar a problemas graves a largo plazo.
- No hay una “talla única” en cuanto a metodologías y herramientas; la elección debe basarse en los requisitos específicos del proyecto y las necesidades del equipo.
- Mantener una documentación detallada y facilitar la comunicación entre los diferentes stakeholders no sólo mejora el proceso de desarrollo, sino que también es vital para el mantenimiento y la escalabilidad futura del sistema.

/ BIBLIOGRAFÍA

Bass, L., Clements, P., & Kazman, R. (2012). Software Architecture in Practice (3rd ed.). Addison-Wesley.

Martin, R. C. (2002). Agile Software Development, Principles, Patterns, and Practices. Prentice Hall.

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

<https://www.infoworld.com/category/software-architecture/>

<https://martinfowler.com/>

<https://www.udacity.com/course/software-architecture-design--ud821>